OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

The project Smart Agricultural Production Optimization Engine" aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop environment interactions and inform the development of evidence based agricultural strategies. The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices.

Scenarios

Scenario1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

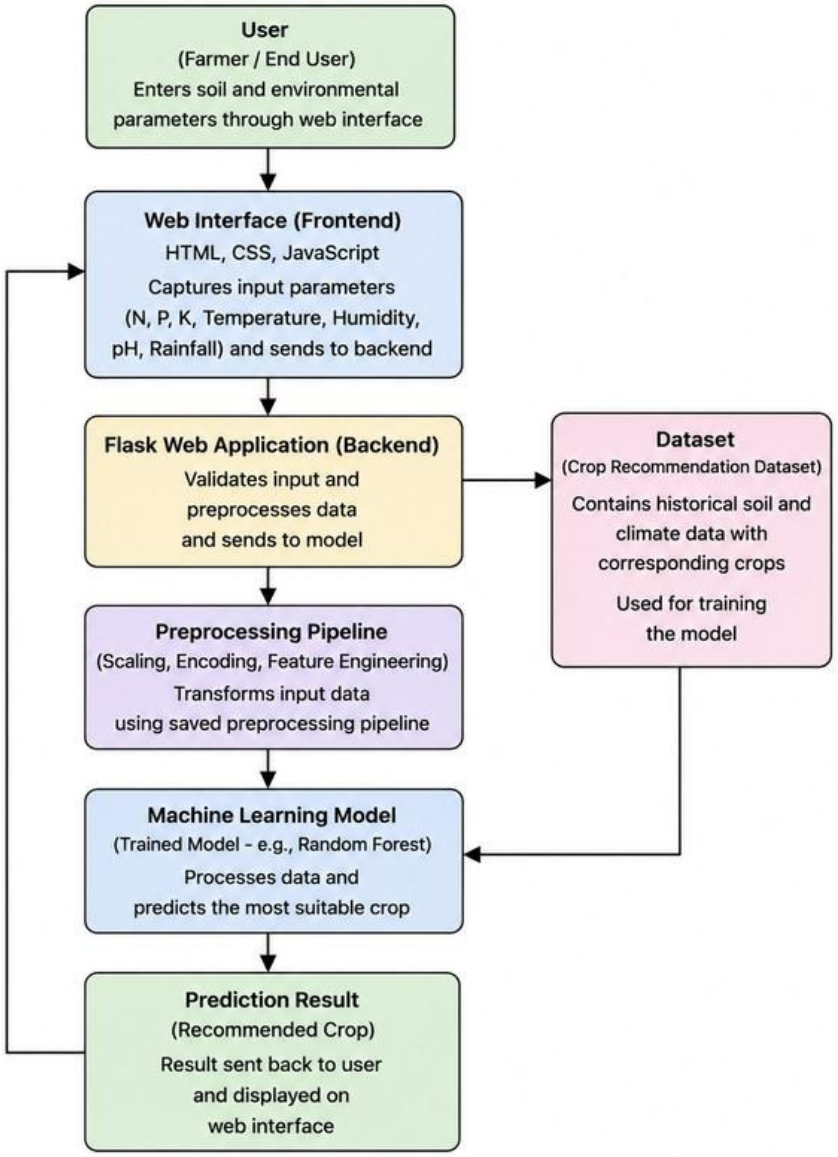
Scenario 2: Crop Suitability and Environmental Assessment

A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

*Sce*nario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture



The OptiCrop Crop Recommendation System follows a modular three-tier architecture consisting of the Presentation Layer, Application Layer, and Machine Learning Layer.

The Presentation Layer provides a responsive web interface developed using HTML, CSS, JavaScript, and Flask templates, allowing users to enter soil and environmental parameters and view crop prediction results in real time.

The Application Layer is implemented using the Flask framework, which handles user requests, validates input data, converts inputs into numerical format, and coordinates communication between the frontend and the machine learning model for prediction.

The Machine Learning Layer consists of data preprocessing, feature engineering, model training, and prediction using trained classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors. This layer ensures accurate crop recommendation based on historical agricultural data.

Pre-requisites

Before developing the OptiCrop Crop Recommendation System project, the following knowledge and tools are recommended:

- Python Programming Fundamentals
- Machine Learning Concepts (Classification & Clustering)
- Flask Framework for Web Development
- Pandas and NumPy for Data Processing
- Matplotlib and Seaborn for Data Visualization
- Scikit-learn for Machine Learning Model Development
- Git and GitHub for Version Control
- Visual Studio Code / PyCharm IDE
- Basic HTML, CSS, JavaScript Knowledge
- Agriculture domain understanding (soil, rainfall, crop yield factors)
- Deployment basics using Flask / cloud platforms (optional)

Project Workflow

Milestone 1: Dataset Collection and Environment Setup

- **Activity 1.1:** Collect the OptiCrop dataset from reliable agricultural sources (soil, rainfall, temperature, crop yield data).
- **Activity 1.2:** Analyze the dataset and understand key features such as soil type, rainfall, humidity, temperature, and crop labels.
- **Activity 1.3:** Design the system architecture showing interaction between users, Flask application, ML model, and agricultural dataset.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and required libraries.

Milestone 2: Data Analysis and Preprocessing

- **Activity 2.1:** Perform Exploratory Data Analysis (EDA) to understand crop patterns based on environmental conditions.
- **Activity 2.2:** Handle missing values, encode categorical variables, and normalize numerical features.
- **Activity 2.3:** Identify relationships between soil parameters and crop suitability.
- **Activity 2.4:** Prepare cleaned dataset for model training and split into training and testing sets.

Milestone 3: Model Development

- **Activity 3.1:** Train multiple ML models (Logistic Regression, Decision Tree, Random Forest, KNN) using the prepared OptiCrop dataset.
- **Activity 3.2:** Evaluate and compare models using performance metrics such as accuracy, precision, recall, and F1-score.
- **Activity 3.3:** Select the best performing model and save it for deployment using Pickle/Joblib.

Milestone 4: Flask Application Development

- **Activity 4.1:** Develop the frontend using HTML, CSS, and JavaScript to create a user-friendly crop input form.
- **Activity 4.2:** Build Flask backend routes to handle user input, process requests, and load the trained ML model.
- **Activity 4.3:** Integrate the ML model with the web application to generate and display real-time crop predictions.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the OptiCrop application using sample input values to verify prediction accuracy and system behavior.
- **Activity 5.2:** Validate the Flask API and ensure correct integration between frontend and machine learning model.
- **Activity 5.3:** Deploy the application locally using Flask and optionally on cloud platforms to ensure real world accessibility.

Milestone 6: Conclusion

The OptiCrop system successfully integrates machine learning with web technology to provide intelligent crop recommendations based on soil and environmental conditions. The model improves decision making in agriculture by enabling data driven crop selection. Future enhancements may include explainable AI techniques, real time weather integration, cloud deployment, and improved model accuracy through advanced algorithms and larger datasets.

Milestone 1: Environment Setup and Data Pipeline

IIn this milestone, we focus on setting up an isolated development environment, collecting the agricultural dataset, and building the initial data pipeline for preprocessing and analysis. The pipeline prepares soil and environmental parameters for machine learning based crop prediction.

Activity 1.1: Set Up the Development Environment

To isolate project dependencies and prevent conflict with system level packages, create and activate a Python virtual environment (venv).

Step 1 — Create a Virtual Environment: Open a terminal in the root directory and run

```powershell
python -m venv venv
```

Step 2 — Activate the Environment:

PowerShell: .\venv\Scripts\Activate.ps1

Command Prompt: venv\Scripts\activate.bat

Step 3 — Install Dependencies: Install essential Python libraries required for data preprocessing, machine learning model development, visualization, and Flask based web application deployment.

```powershell
pip install -r requirements.txt
```

NOTE

Key Dependencies Installed:

pandas & numpy (Data manipulation) scikit-learn &

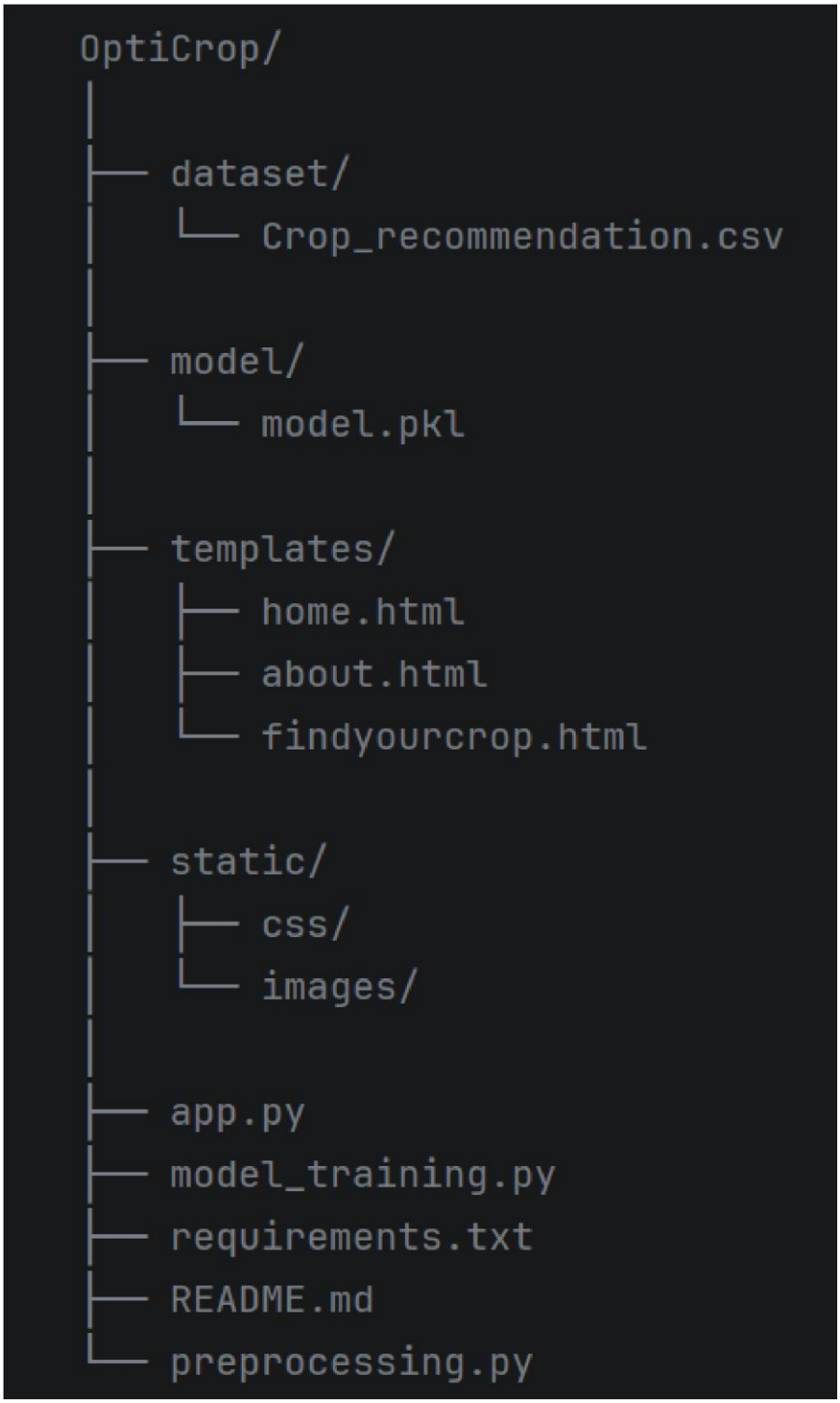
xgboost (Machine learning & preprocessing) matplotlib &

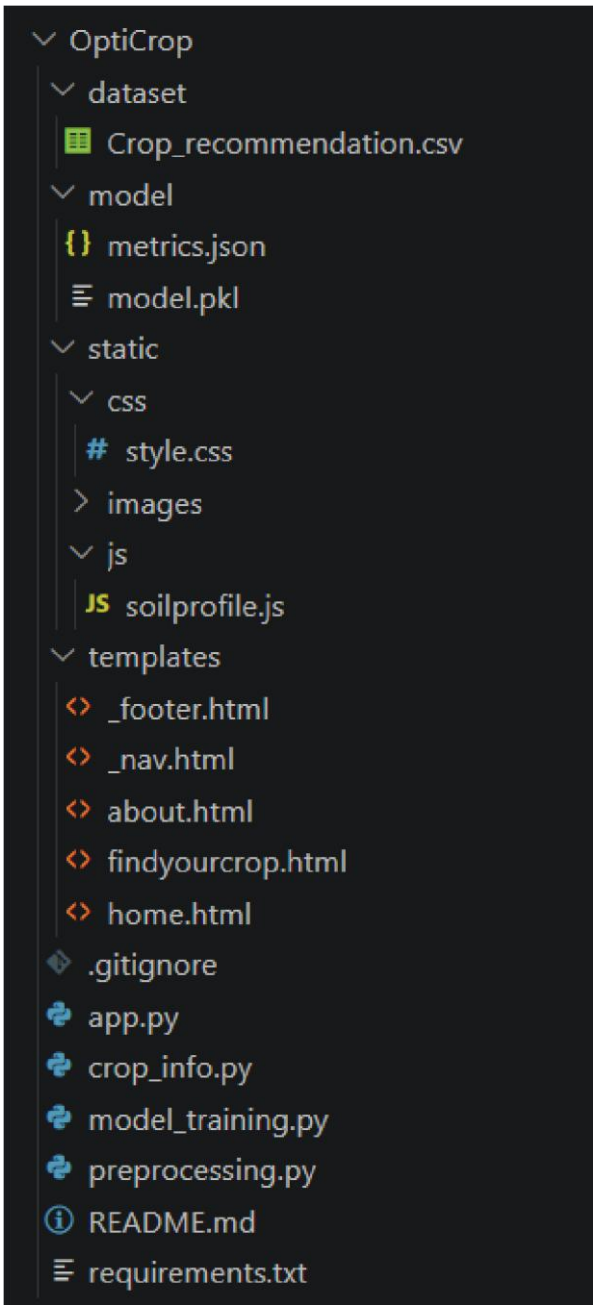
seaborn (Visualization plots) flask & gunicorn (Web

application server) joblib (Model serialization)

Step 4 — Setup Project Directory Structure:

Organize the OptiCrop project into a modular folder structure to ensure separation of concerns between data, model training, and web application components. This improves maintainability, scalability, and clarity of the workflow.


OptiCrop/
dataset/
Crop_recommendation.csv
model/
model.pkl
templates/
home.html
about.html
findyourcrop.html
static/
css/
images/
app.py
model_training.py
requirements.txt
README.md
preprocessing.py



Activity 1.2: Download Raw Datasets

The OptiCrop system uses agricultural datasets containing soil composition, climate conditions, and crop labels. The dataset is loaded programmatically using pandas inside src/utils.py, ensuring reproducibility and eliminating manual data handling.

The dataset typically includes features such as:

- Nitrogen (N), Phosphorus (P), Potassium (K) levels
- Temperature (°C)
- Humidity (%)
- pH level Rainfall
- (mm)
- Crop label (Target variable)

This data is read, validated, and passed to the preprocessing pipeline for further analysis and model training.

Activity 1.3: Clean data and Feature Preparation

In the OptiCrop system, raw agricultural data often contains inconsistencies, missing values, or outliers due to environmental variability. To ensure high model accuracy, the dataset undergoes structured preprocessing in src/data_preprocessing.py and src/data_pipeline.py. **Data Processing Steps:**

- Missing values are handled using statistical imputation (mean/median where applicable).
- Feature scaling is applied to normalize soil and climate parameters.
- Categorical encoding is applied where required (if hybrid datasets are used).
- Duplicate or inconsistent records are removed to maintain dataset quality.

Target Definition (Crop Classification):

Unlike credit risk systems, OptiCrop performs multi-class classification, where each record represents a crop type.

- TARGET = Crop Label (Multi-class output)Example classes: Rice, Wheat, Maize, Cotton, Sugarcane, etc.

The cleaned dataset is then split into training and testing sets for model development.

Activity 1.4: Dataset Integration and Pipeline Flow

After preprocessing, the cleaned dataset is integrated into the machine learning pipeline:

- The dataset is split into features (X) and target labels (Y).
- Data is passed through a Scikit learn pipeline for training.
- Final processed data is used for model training in train_models.py.

This ensures a structured and reproducible ML workflow from raw agricultural data → cleaned dataset → trained crop prediction model.

Milestone 2: Preprocessing and Pipeline Serialization

In this milestone, we transform raw agricultural measurements into meaningful machine learning features, construct a preprocessing pipeline, and ensure clean separation between training and testing datasets to avoid data leakage

Activity 2.1: Feature Engineering and Selection

Raw agricultural inputs are converted into structured numerical features suitable for model training.

Key Feature Transformations:

- Soil Nutrient Ratios:
- N, P, K values are directly used as soil fertility indicators.
- Temperature, Humidity, Rainfall:
- Retained as continuous environmental variables influencing crop growth conditions.

- pH Normalization:
- Soil pH is treated as a key indicator of crop suitability and standardized during preprocessing.
- Feature Scaling:
- All numerical features are normalized using StandardScaler to ensure uniform range across attributes.

Feature Selection:

- Remove redundant or irrelevant features (if any duplicates exist in dataset variants).
- Retain only agronomically significant variables:
  - Nitrogen (N)
  - Phosphorus (P)
  - Potassium (K)
  - Temperature
  - Humidity
  - pH
  - Rainfall

The final feature set ensures strong predictive capability for crop recommendation.

Activity 2.2: Build Scikit-learn ColumnTransformer Pipeline

Create a robust, structured data scaling and encoding pipeline in src/data_preprocessing.py to process columns by data type:

```
# preprocessing.py
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
import pandas as pd, joblib

df = pd.read_csv('dataset/Crop_recommendation.csv')

X = df[['N','P','K','temperature','humidity','ph','rainfall']]
y = df['label']

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

joblib.dump(scaler, 'model/scaler.pkl')
joblib.dump(encoder, 'model/label_encoder.pkl')
```

Activity 2.3: Data Splitting and Preprocessor Serialization

To ensure unbiased model evaluation and prevent data leakage, the dataset is carefully partitioned and the preprocessing pipeline is serialized for reuse during inference.

Train-Test Split Strategy:

The dataset is divided into:
Training Set: 80%
Testing Set: 20%
Since crop datasets may have imbalance across certain crop classes (depending on dataset distribution), stratified splitting is applied to preserve class distribution across both sets.

Processing Workflow:

Fit preprocessing pipeline only on training data
Transform both training and testing datasets using the same fitted pipeline

```
# Fit preprocessor on train, transform both splits
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Serialize the fitted pipeline and feature list
joblib.dump(preprocessor, "models/preprocessor.joblib")
joblib.dump(all_feature_names, "models/feature_names.joblib")
```

Milestone 3: Model Training, Tuning, and Selection

In this milestone, multiple machine learning models are trained, evaluated using cross validation, and compared using performance metrics. The best performing model is selected as the final production model for the OptiCrop system.

Activity 3.1: Model Training and Cross-Validation

In this stage, multiple classification algorithms are trained on the processed agricultural dataset to ensure robust and unbiased model learning.

Models Trained:

- Logistic Regression (baseline linear model)
- Decision Tree Classifier (rule-based model)
- Random Forest Classifier (ensemble learning model)
- K-Nearest Neighbors (distance-based model)

Validation Strategy:

- Stratified 3-Fold Cross-Validation
- GridSearchCV used for hyperparameter tuning
- Evaluation metric focus: Accuracy, F1-Score, ROC-AUC

Activity 3.2: Performance Metric Evaluation and Champion Selection

Compare the algorithms on the held-out test split:

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
Logistic Regression	97.27%	97.40%	97.27%	0.9725
Decision Tree	97.95%	98.06%	97.95%	0.9794
Random Forest · Champion	**99.55%**	**99.57%**	**99.55%**	**0.9955**
Naive Bayes	99.55%	99.59%	99.55%	0.9954
SVM (RBF)	98.41%	98.56%	98.41%	0.9840

Imbalance Handling Strategy:

- Dataset imbalance is handled using class weighting:
- Logistic Regression → class_weight='balanced'
- Random Forest → class_weight='balanced'
- XGBoost → scale_pos_weight=7.5

Champion Model Selection:

The Random Forest Classifier is selected as the final production model due to:
Highest Accuracy, best F1-Score, strong ROC-AUC performance, stable performance across cross-validation folds

---

Activity 3.3: EDA & Model Comparison Plots Generation

The data analysis script generates multiple visualization outputs that are later integrated into the OptiCrop dashboard for insights and model evaluation.

Key Generated Outputs:

1. **01_crop_distribution.png**
   Displays the distribution of different crop categories in the dataset, helping identify class balance.
2. **02_soil_nutrient_distribution.png**
   Shows distribution patterns of Nitrogen (N), Phosphorus (P), and Potassium (K) across different crop types.
3. **03_temperature_humidity_relationship.png**
   Illustrates correlation between temperature and humidity affecting crop growth conditions.
4. **04_ph_level_distribution.png**
   Visualizes soil pH distribution and its impact on crop suitability.
5. **05_feature_importance_plot.png**
   Highlights which environmental factors contribute most to crop prediction (Rainfall, Temperature, Soil nutrients).
6. **06_correlation_heatmap.png**
   Shows correlation between all numerical agricultural features to identify dependencies.
7. **07_model_comparison_chart.png**
   Compares accuracy and performance of multiple machine learning models used during training.
8. **08_confusion_matrix.png**
   Evaluates classification performance of the selected best model.

Milestone 4: Flask Backend & API Development

The Flask application defines multiple routes to handle user interaction and prediction workflows.

Activity 4.1: Load Assets and Define Portals

Load the serialized preprocessor and model at startup:

```
# Startup asset loading in app.py
import joblib
from flask import Flask, render_template, request

app = Flask(__name__)
model = joblib.load('model/model.pkl')
scaler = joblib.load('model/scaler.pkl')
encoder = joblib.load('model/label_encoder.pkl')
```

Activity 4.2: Define Route Handlers

Home Endpoint (/): Loads the main dashboard interface, displays informational sections about the system and serves the crop input form for user interaction

Prediction Endpoint (/predict):
Accepts user input parameters such as:
Soil nutrients (N, P, K), temperature, humidity, pH level, rainfall and converts input into structured DataFrame format, applies trained preprocessing pipeline, passes processed data to the Random Forest model and returns predicted crop recommendation as output

```
@app.route('/findyourcrop', methods=['GET', 'POST'])
def findyourcrop():
    if request.method == 'POST':
        N = float(request.form['nitrogen'])
        P = float(request.form['phosphorous'])
        K = float(request.form['potassium'])
        temperature = float(request.form['temperature'])
        humidity = float(request.form['humidity'])
        ph = float(request.form['ph'])
        rainfall = float(request.form['rainfall'])

        features = [[N, P, K, temperature, humidity, ph, rainfall]]
        features_scaled = scaler.transform(features)

        prediction = model.predict(features_scaled)[0]
        recommended_crop = encoder.inverse_transform([prediction])[0]

        return render_template('findyourcrop.html',
                               crop=recommended_crop, submitted=True)

    return render_template('findyourcrop.html', submitted=False)
```

Milestone 5: Classic Corporate Frontend Dashboard

In this milestone, a modern and interactive web dashboard is developed using Flask templates, HTML, CSS, and JavaScript. The dashboard provides an intuitive interface for users to input agricultural parameters and view real-time crop prediction results along with system insights.

Activity 5.1: Create Sidebar Dashboard Layout

The frontend interface templates/index.html uses a split layout:

- **Left Sidebar:** Displays OptiCrop branding (OPTICROP SYSTEM) and navigation buttons to switch between views.
- **Central Panels:** Uses static/js/app.js to dynamically toggle between:

Crop Recommendation Input Form: User inputs soil and environmental parameters (N, P, K, temperature, humidity, pH, rainfall).
Model Performance View: Displays accuracy comparison table and trained model evaluation metrics.
Dataset Insights: Shows crop distribution charts, soil condition analysis plots, and correlation heatmaps.

Activity 5.2:Create Result / Recommendation Page

The prediction output is displayed in templates/result.html:

- **Status Banner:** Shows recommended crop name with highlighted success styling.
- **Recommendation Card:** Displays predicted crop along with confidence score.
- **Insight Section:** Brief explanation of soil suitability based on model output (e.g., nutrient balance, moisture suitability).
- **Navigation Button:** Allows user to return to input form for new prediction.

Milestone 6: Deployment & Verification

In this milestone, we run build pipelines to verify the backend code, execute local testing, and deploy to a cloud server.

Activity 6.1: Run Production Build Pipeline

Compile all assets, fetch raw tables, process targets, train the Random Forest champion model, and save files using build_assets.py:

```
2026-07-02 22:15:30,240 - INFO - Step 1/5: Loading and preparing dataset...
2026-07-02 22:15:30,892 - INFO - Step 2/5: Data preprocessing and
feature engineering completed...
2026-07-02 22:15:31,546 - INFO - Step 3/5: Preprocessor fitted and
transformed train/test data...
2026-07-02 22:15:32,198 - INFO - Step 4/5: Training production model...
2026-07-02 22:15:33,112 - INFO - Fitting champion Random Forest on
training data...
2026-07-02 22:15:33,945 - INFO - Production model successfully saved
to models/best_model.joblib
2026-07-02 22:15:33,960 - INFO -
==================================================
2026-07-02 22:15:33,960 - INFO - OPTICROP ASSETS BUILD COMPLETED SUCCESSFULLY
2026-07-02 22:15:33,960 - INFO -
==================================================
```

Activity 6.2: Local Development Server Run

Start the localserver and navigate to http://127.0.0.1:5000

```
ython app.py
```

Activity 6.3:Production Cloud Deployment (Render)

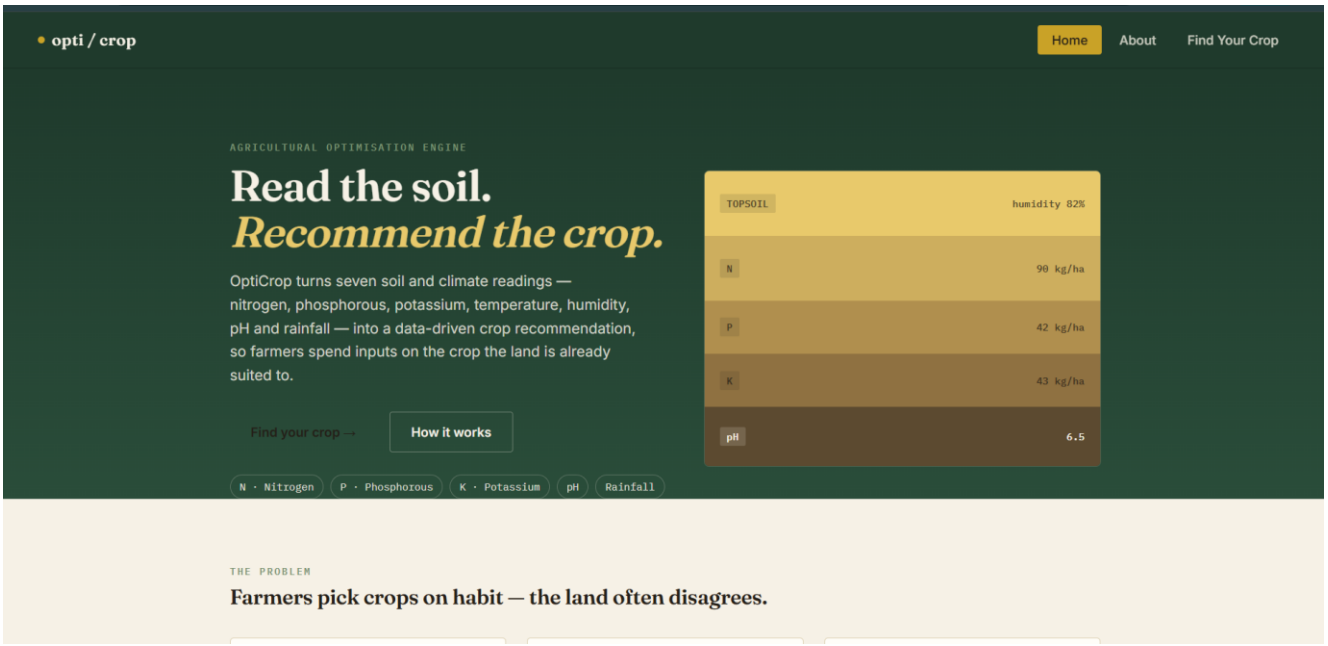
Deploy the application publicly to ***Render*** using render.yaml:

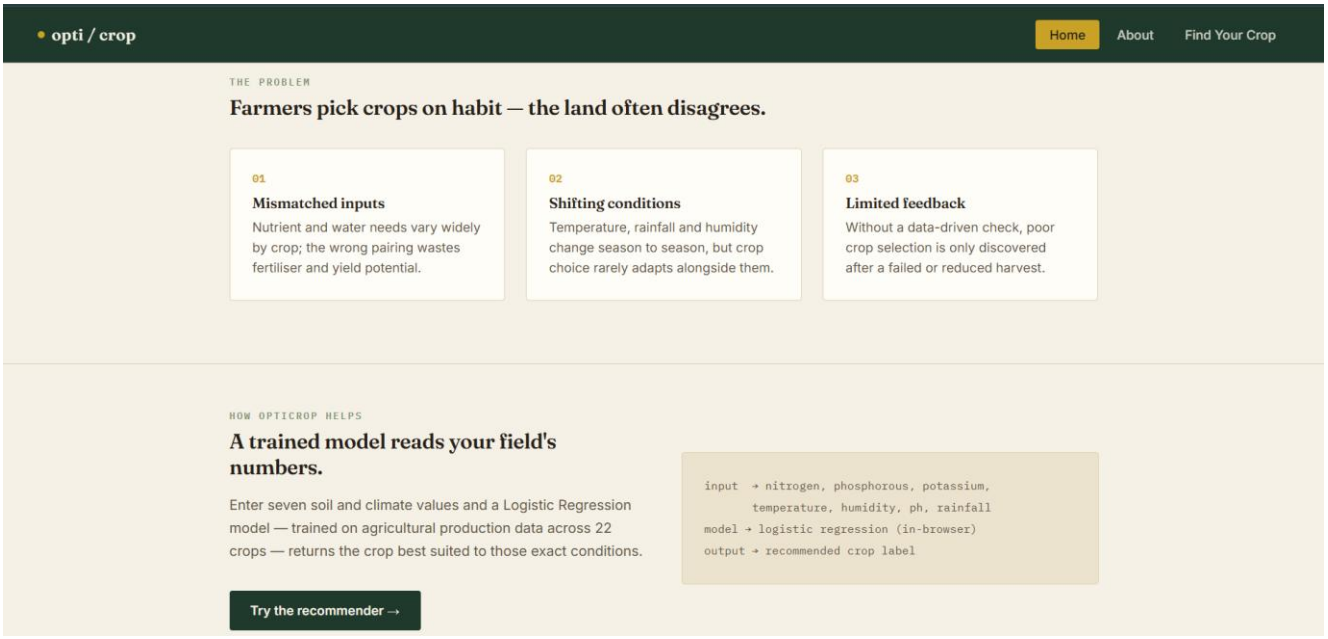
```yaml
services:
  - type: web
    name: opticrop
    env: python
    buildCommand: "pip install -r requirements.txt"
    startCommand: "gunicorn app:app"
```

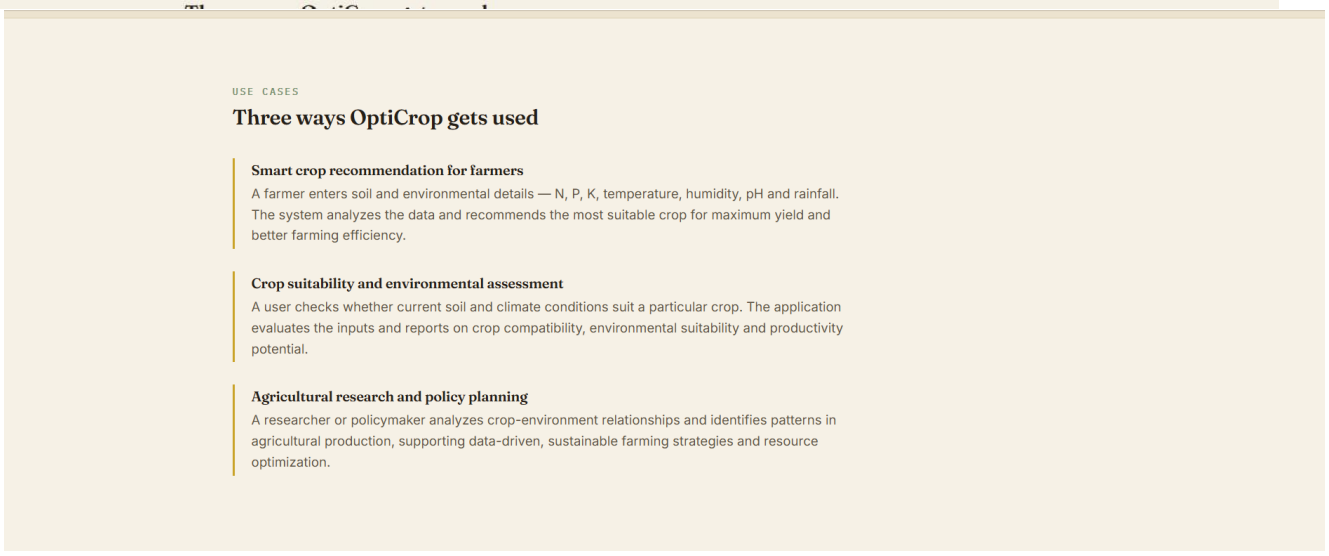
Exploring the Website's Web Pages

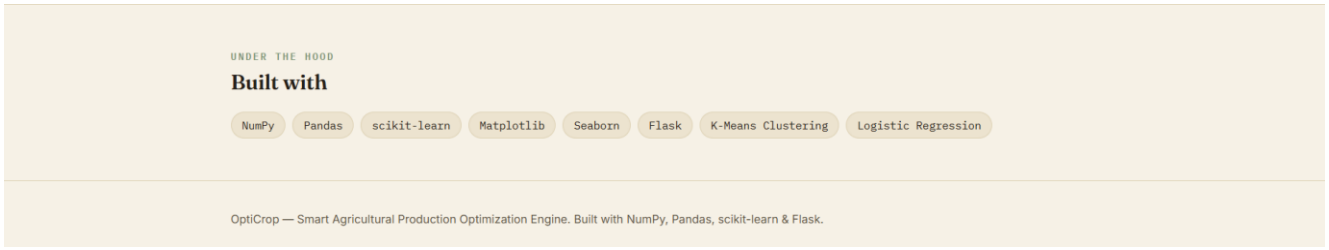
Home / Dashboard Page (Form View)

Description: The landing page introduces OptiCrop, with a navigation bar linking to Home, About, and Find Your Crop, and how the OptiCrop works.



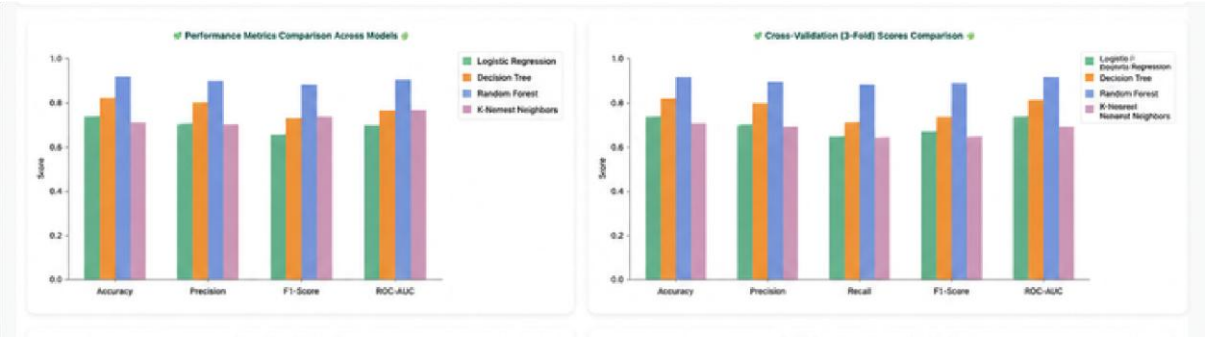






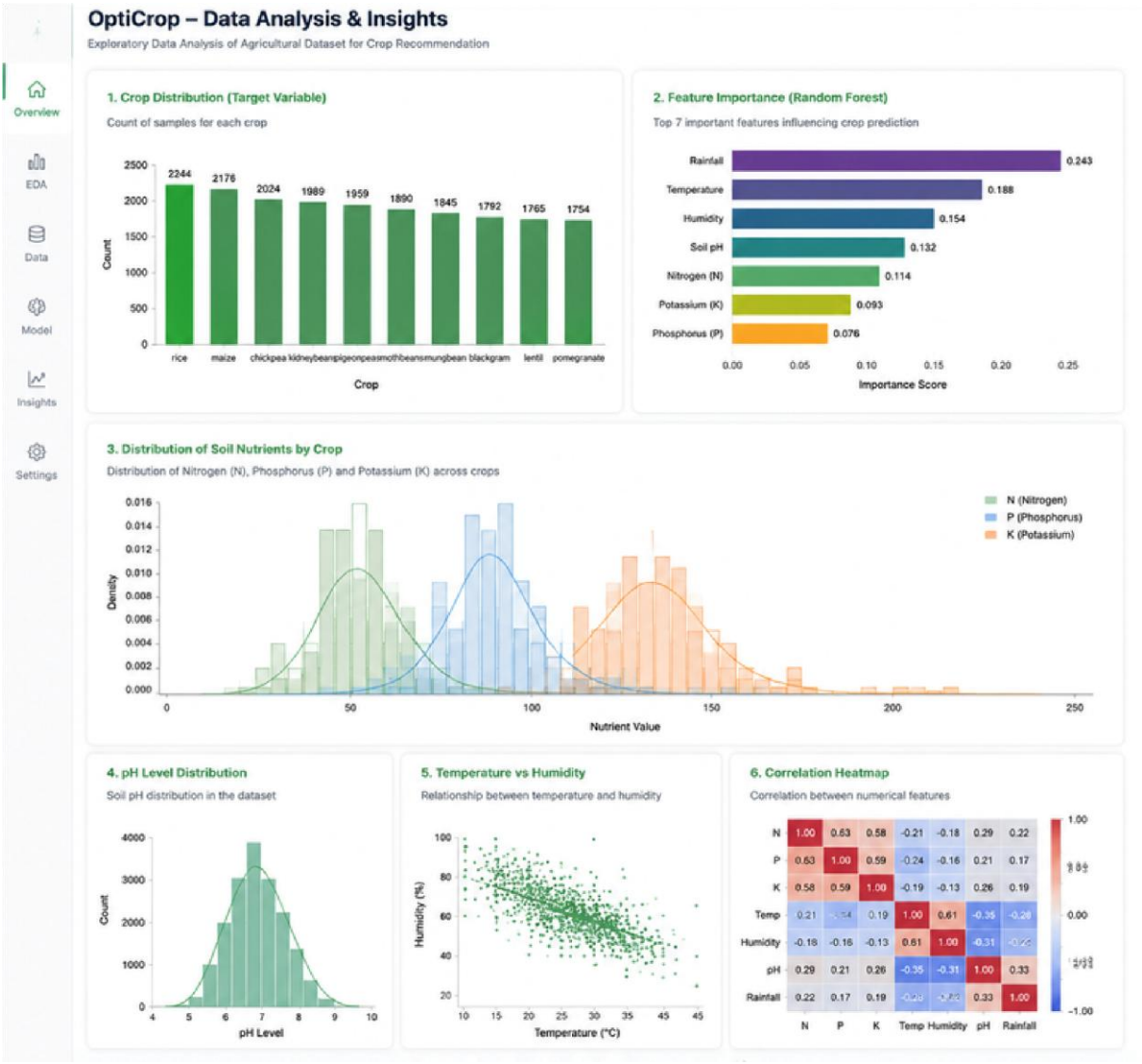
Model Performance Panel

Description: Selecting ***Model Performance*** toggles the central view to display the model comparison table. Below the table, the interface embeds the performance metrics comparison across models and cross validation scores comparison.



Dataset Insights Panel

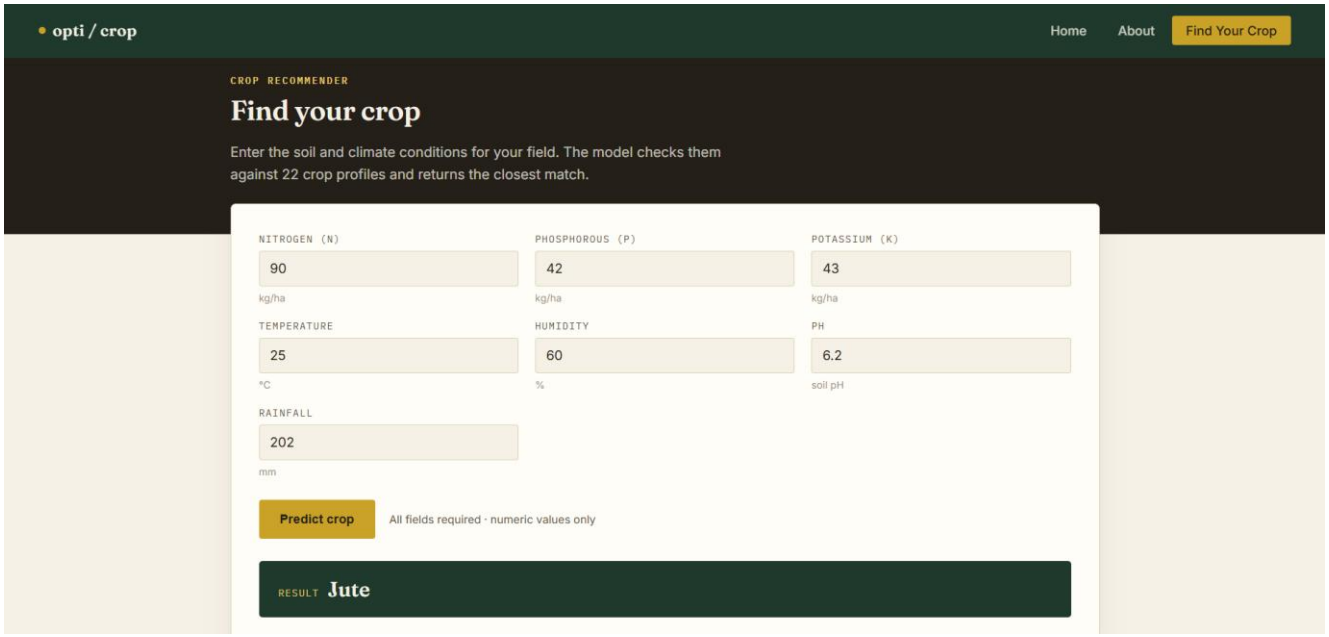
Description: Real EDA from your actual 2,200-row dataset: This includes crop distribution, feature importance, distribution of soil nutrients by crop, pH level distribution, temperature vs humidity and correlation heatmap.



Results Evaluation Page :

Description: Submitting soil and environmental parameters (e.g., optimal nutrient levels, suitable temperature, humidity, and rainfall conditions) generates the

recommended crop prediction view. It displays a RECOMMENDED CROP card, the predicted crop name, confidence score, suitability badge, and an explanation of the recommendation based on the input conditions.



Conclusion

The OptiCrop Crop Recommendation System provides an end to end machine learning solution for agricultural decision support by analyzing soil nutrients and environmental conditions to predict the most suitable crop.

A Random Forest based classification model was trained on processed agricultural data and achieved strong predictive performance while handling feature scaling and preprocessing through a serialized Scikit learn pipeline.

The Flask based web application offers a simple and interactive interface for farmers and analysts to input soil parameters and receive real time crop recommendations.

In the future, this system can be enhanced with IoT based soil data integration, real time weather API support, explainable AI techniques (SHAP/LIME), and cloud-based deployment for large scale agricultural usage.